

CSCE 606 SOFTWARE ENGINEERING · MODULE 09 · LECTURE NOTES

Code Review

The highest-leverage practice, done badly almost everywhere.

MODULE OUTCOMES

- State what review is actually for, in priority order.
- Review effectively: what to look for and what to ignore.
- Give feedback that improves code without damaging people.
- Size changes so review is possible.
- Explain what the evidence says about review effectiveness.

1 What review is for

Purpose	Value
Knowledge sharing	After review, at least two people understand the change. This is how a team avoids code that only one person can maintain, and it is what makes departures survivable. Probably the largest benefit.
Maintainability	A second reader is empirical evidence that the code can be understood by someone who did not write it — which is the condition Module 01 identified as determining lifetime cost.
Design feedback	The cheapest moment to change an approach is before it has been built on. After merge the cost rises steeply.
Correctness	Review does find defects. Tests find more, and find them repeatedly and automatically.
Consistency	Real, and almost entirely automatable. See below.
Mentoring	In both directions. A junior reviewing senior code learns the codebase faster than by reading it alone.

AUTOMATE EVERYTHING MECHANICAL

Formatting, import ordering, naming conventions, lint rules, unused variables, obvious null dereferences — a machine checks all of these faster, more consistently, and without anyone's feelings being involved.

Every human comment spent on them is attention not spent on design, correctness, or clarity, which are the things only a human can assess.

Run the formatter and the linter in CI and make them blocking. Then the style question is settled once, in a configuration file, rather than renegotiated in every pull request forever.

Which formatter does not matter. Pick a defensible default, commit the configuration, and decline to discuss it again. The time saved by not having that argument repeatedly exceeds any benefit from winning it.

2 Reviewing well

Review in this order. The ordering matters because effort spent at the bottom is wasted if something at the top is wrong.

- 1 **Should this change exist?** Does it solve the right problem? Is the problem worth solving? Is there a simpler approach that makes it unnecessary? This is the cheapest possible place to catch a misdirected effort, and the most expensive place to miss one.
- 2 **Is the approach right?** Does it fit the system's existing structure? Does it put responsibility in the right place (Module 03)? Does it introduce coupling that will be expensive?
- 3 **Is it correct?** Edge cases, boundary values, error paths, concurrency, resource cleanup. Use Module 06's partitioning as a checklist.
- 4 **Is it adequately tested?** Not 'are there tests' but do the tests test *behaviour* (Module 05), and do they cover the cases that worry you?
- 5 **Is it readable?** Will someone encountering this in two years understand it? If you had to reread a section three times, say so — that is data.
- 6 **Mechanical issues. The machine does this.** If you are here, something is misconfigured.

CHANGE SIZE IS THE DOMINANT VARIABLE

This is the best-supported empirical finding about code review, and it is consistently ignored.

Defect detection rates fall sharply as change size grows. Beyond roughly 400 lines, reviewers find very little — the review becomes a skim followed by an approval, because sustained careful attention does not extend that far.

A 2,000-line pull request does not receive four times the scrutiny of a 500-line one. It receives less scrutiny in total, because the reviewer disengages.

Under 200 lines is where review is genuinely effective. This makes keeping changes small primarily the **author's** responsibility, and it is the single most useful thing an author can do for review quality.

2.1 What the author owes the reviewer

- **Keep it small.** Several small pull requests reviewed properly beat one large one approved without reading.
- **Separate refactoring from behavioural change** (Module 07). A diff mixing a rename across forty files with a logic change is unreviewable: the logic change is invisible in the noise.
- **Write the description.** What changed, why, how to verify it, and what you are uncertain about. The reviewer has a fraction of your context.
- **Review your own diff first.** Reading your change as a diff rather than as an edit surfaces a surprising number of problems, and finding them yourself is cheaper than a round trip.
- **Direct attention.** 'The locking in handler.rs is the part I am least sure about' is worth more than any amount of general politeness.

3 Giving feedback

This determines whether a review culture is sustainable. Reviews that feel like attacks produce defensive authors, larger batches to reduce exposure, and eventually rubber-stamping.

Instead of	Write	Because
'This is wrong.'	'What happens if items is empty?'	The specific question is actionable, and may have an answer.
'Why would you do it this way?'	'Have you considered X? It would avoid Y.'	The first reads as an accusation; the second as a contribution.
'Bad variable name.'	'I read count as a running total — is it actually a limit?'	Reports the confusion, which is the evidence, rather than issuing a verdict.
'Needs tests.'	'Could we add a case for the timeout path?'	Specific and immediately actionable.
'nit: inconsistent spacing'	Nothing.	The formatter handles it. If it is not handled, fix the configuration, not the pull request.

QUESTIONS OUTPERFORM VERDICTS, AND ARE MORE HONEST

A question invites an explanation. Often the explanation is good, and the reviewer has learned something: the author spent days in this code and has context the reviewer does not.

A verdict invites defence. The exchange becomes about who is right, and the code does not improve while that is being settled.

The honesty point matters independently. A reviewer reading a diff for ten minutes genuinely knows less than the author. Phrasing as a question reflects the actual epistemic position rather than performing politeness.

Review the code, never the person. 'This function allocates on every call' rather than 'you allocate on every call'. The difference is trivial in text and substantial in how it is received — and it costs nothing.

3.1 Calibrate every comment

Level	Meaning	How to mark it
Blocking	Correctness, security, data loss, or a design problem that will be expensive to undo.	State it plainly: 'This needs to change before merge, because...'
Suggestion	An improvement the author may take or leave.	Prefix nit: or optional:. Mean it — if it is not optional, do not label it so.
Question	You do not understand something.	Ask. These are often the most valuable comments: if you cannot follow it after ten minutes, the next reader will not either.

Label every comment. An unlabelled suggestion reads as a requirement, and authors rewrite perfectly good code to satisfy a preference nobody intended to impose. This single habit removes most review friction.

THE APPROVAL STANDARD

Google's documented rule: **approve when the change definitely improves the overall health of the codebase, even if it is not perfect.**

Holding out for perfection blocks progress, exhausts authors, and — most damagingly — incentivises larger changes. If each review round trip costs two days, authors batch work to reduce the number of round trips, which makes review less effective, which makes reviewers more anxious, which makes them stricter.

Approve with comments for anything non-blocking and let the author decide. A culture where approval is rare and hard-won produces fewer, larger, more poorly reviewed changes — the opposite of what the strictness was for.

4 What the evidence supports

Claim	Status
Code review finds defects.	Well supported , though estimates of how many vary enormously across studies and contexts.
Review effectiveness falls sharply with change size.	The strongest and most consistent finding. Act on this one.
Knowledge transfer is a major benefit, possibly the largest.	Well supported by practitioner studies and consistent with how teams report using it.
A second reviewer adds substantial value over the first.	Weakly supported. Most studies find sharply diminishing returns after one careful reviewer.
Review substitutes for testing.	No. They detect different categories of problem, and review does not repeat itself automatically on every future change.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Google's Code Review Developer Guide (free)	The best free material on this, from both the reviewer's and the author's side. The 'approve when it improves things' standard comes from here.

Resource	What it covers for this module
Software Engineering at Google — Chapter 9 (Code Review)	Why they review, what they found it actually buys, and the data behind it.
SmartBear — Best Kept Secrets of Peer Code Review (free summary)	The source of the frequently cited size and duration findings.
Conventional Comments	A light convention for labelling comments by type and severity. Adopting it removes most review friction for free.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Review an open pull request in a project you do not maintain. Write the review privately first, then compare against the actual reviewers' comments. What did you miss, and what did they miss?
- 2 Take one of your own changes larger than 500 lines and split it into pull requests of under 200 lines each, in a sensible order. Record how long it took and whether the split revealed anything.
- 3 Review your own last five commits as diffs rather than as remembered edits. Record what you find.
- 4 Take five review comments you have written or received and reclassify each as blocking, suggestion, or question. Rewrite any that were miscalibrated.
- 5 Configure a formatter and linter for a project and make them blocking in CI. Count the review comments they would have made unnecessary over the last twenty pull requests.
- 6 Find a review conversation in an open-source project that went badly. Identify the specific comment that escalated it and rewrite it.
- 7 Measure review latency in a project you can observe: time from pull request opened to first substantive comment, plotted against change size.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is code review's largest benefit, and where does defect-finding rank?
- 2 Why should mechanical issues never appear in human review comments?
- 3 List the six things to look at in review, in order, and say why the order matters.
- 4 What happens to defect detection above 400 lines, and whose responsibility is change size?
- 5 Give two reasons questions work better than verdicts in review.
- 6 What are the three comment levels, and what goes wrong if they are unlabelled?

7 What is the approval standard, and what does a culture of rare approval produce?